Composite Design Pattern with a Different Example

Overview

The Composite design pattern is an organizational pattern that facilitates the representation of part-whole hierarchies through a tree structure. It allows composite objects to be treated uniformly with individual objects, simplifying the client interface when interacting with different objects within the hierarchy.

Key Concepts

- **Component:** The interface or abstract class that standardizes operations for both simple and complex objects within the tree.
- **Leaf:** Implements the component interface and represents objects that do not have sub-objects in the hierarchy.
- **Composite:** Also implements the component interface and represents complex elements that have children. It stores child components and implements methods to add and remove them.

Example: Graphic Objects

Imagine a graphic design software where objects can be manipulated individually or as grouped collections.

- **Component:** Graphic
- **Leaf:** Shape (e.g., Circle, Rectangle)
- **Composite:** Group of Shapes

Structure

- The Graphic component provides common methods such as `draw()`, `scale()`, and `move()`
- Shapes are leaves that implement these methods specifically for graphical shapes.
- Group of Shapes acts as a composite that can contain any number of child graphics, each potentially being either a simple shape or another group.

Example in Practice

In the interface of a graphic design software:

- **Component:** Graphic

```java
public interface Graphic { void draw();
void move(int x, int y); void scale(float factor);
}
```

- **Leaf:** Rectangle

```java
public class Rectangle implements Graphic { void draw() {
      System.out.println("Drawing a rectangle.");
}
void move(int x, int y) {
      System.out.println("Moving the rectangle.");
              }
void scale(float factor) {
      System.out.println("Scaling the rectangle.");
              }
          }
```

- **Composite:** Group

```java
public class Group implements Graphic {
private List<Graphic> children = new ArrayList<>();

          void draw() {
                  for (Graphic child : children) { child.draw();
              }
          }
          void move(int x, int y) {
                  for (Graphic child : children) { child.move(x, y);
              }
          }

     void scale(float factor) {
             for (Graphic child : children) { child.scale(factor);
         }
     }
         void add(Graphic graphic) { children.add(graphic);
     }
         void remove(Graphic graphic) { children.remove(graphic);
     }
 }
```

Benefits

- **Uniformity:** Simplifies the operation on both individual objects and compositions.
- **Flexibility:** Makes it easier to manage and modify complex compositions.
- **Transparency:** Clients interact with all objects in the composite structure uniformly through the Component interface.

Figures and Diagrams (Hypothetical Description)

Consider a UML diagram showing:

- A Graphic interface with methods like `move(),draw(), scale().` .
- Rectangle and Circle as leaves inheriting from Graphic, each implementing the methods.
- Group as a composite also inheriting from Graphic and showing aggregation links to multiple Graphic elements indicating a composition relationship.

This example illustrates how the Composite design pattern is applicable in scenarios where part-whole hierarchies are suitable, such as graphical user interfaces or file systems.